LFCS Additional Essential Commands Topics

Basic Git Operations

Andrew Mallett
Author and Trainer

@theurbanpenguin | www.theurbanpenguin.com

Overview

Through this course:

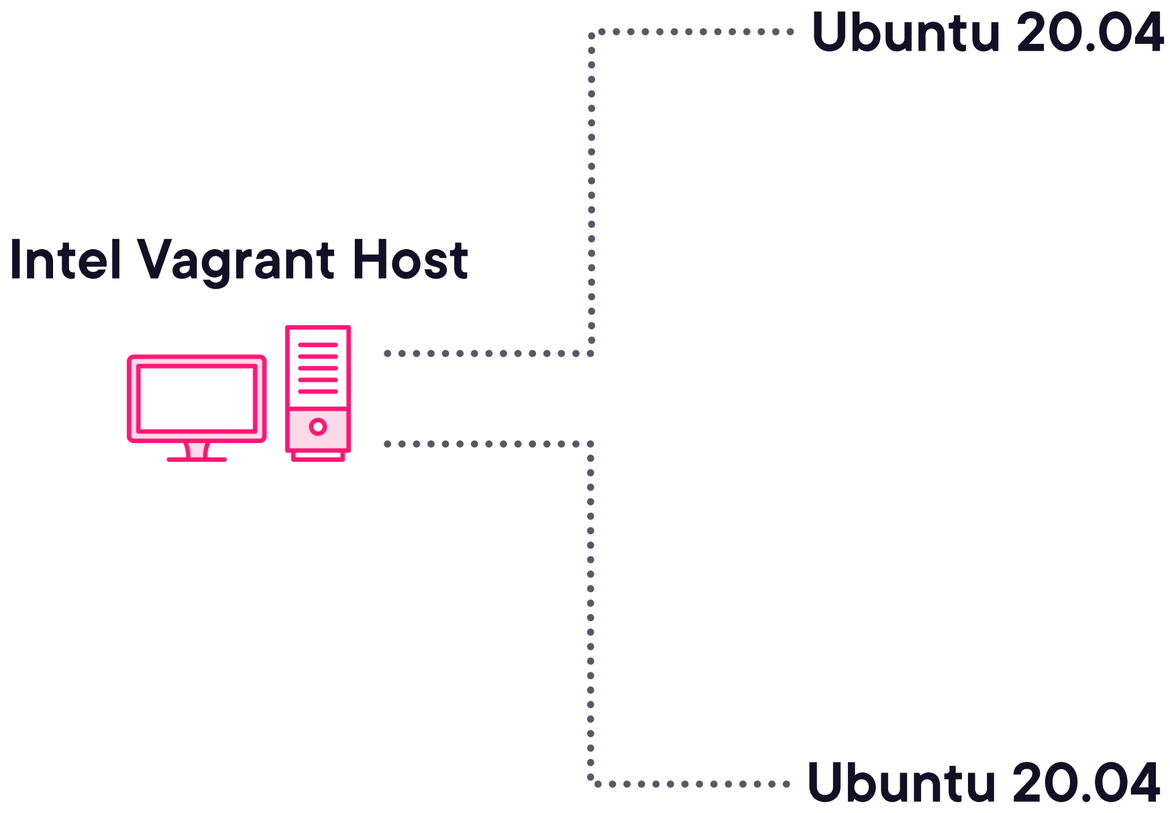
- Git
- Systemd unit files
- OpenSSL

This Module

- Cloning public git repositories
- SSH Key based authentication
- Creating SSH git repository
- Creating branches
- Push to repo
- Merge branches

Lab Systems

https://github.com/theurbanpenguin/lfcs



Working with

GIT

Git is an essential way of distributing data in an open source world. For Admins and Devops, you need the basics

Git:

- Developed by the father of Linux, Linus Torvalds
- Versioning system working best with text files
- Features include branches and tags

```
$ cd
$ sudo apt update && sudo apt install -y git
$ git clone https://github.com/theurbanpenguin/lfcs
$ cd ~/lfcs
$ git status
$ git branch
$ git branch -a
$ git pull
```

Basic Git Clone of Public Repository

Course files are stored in a public git repository, this includes the Vagrantfile that can be used to build the lab systems. By default, the main branch is synchronized, later you will learn more about branching and tagging

Demo

Cloning Git Repositories:

- Ensure git is installed
- Clone course repository (repo)
- Check the status and branch

```
$ git config --list
$ git config --global --list
$ git config --global user.email "tux@example.com"
$ git config --global user.name "Tux Penguin"
$ git config --global --list
$ git config --list
```

Git Configuration

There is a database (.git/) that stores version control information for the git repository. There is also global configuration in the user's home directory and .gitconfig file. To commit changes, even locally we need the user information to be available

Demo

Basic Git Configuration:

- email
- user name

```
$ mkdir ~/myproject ; cd ~/myproject
$ git init ; git status
$ echo "My first line" > myfile ; git status
$ git add .
$ git commit -m "First commit"
$ git show
```

Even Locally: Version Control Can be Useful

Even if you are not going to collaborate on your projects a local git repo will allow you to revert changes or maintain different versions of the same file

```
$ echo "Hello world!" > myfile
$ git add . ; git commit -m "Added text"
$ git log --oneline
$ git revert --no-commit -m1 e778753..HEAD
$ git status
$ cat myfile
$ echo "Hello world!" >> myfile
$ git add . ; git commit -m "Initial text corrected"
```

Using Git: Reverting Changes

As a version control system, git allows us to revert changes to previously committed (saved) versions. Ideally, we would create a development branch for any new code. But even if we haven't, we can still revert. Choose the correct commit identifier to rollback to your previous changes

Demo

Local Git Operations:

- git commit
- git revert

```
$ sudo useradd -m -s /bin/bash git
$ echo 'git:[Password1]' | sudo chpasswd
$ ssh-keygen -N "" -f ~/.ssh/id_rsa
$ ssh-copy-id git@localhost
```

Create Local Git User and SSH Keys

To allow for collaboration on a project git can use SSH. Create a user account for git and allow key based authentication. Having the SSH Server and key-based authentication set up, you have a reliable transport to use git for collaboration

```
$ ssh git@localhost
$ mkdir -p git/project1.git
$ cd git/project1.git
$ git init --bare
$ exit
```

Create an Empty Repo as Git User

You can now connect as the git user and create a project directory with a .git suffix. To initialize the repo without content, use git init --bare.

```
$ git clone git@localhost:~/git/project1
$ cd project1
$ vim hello.sh
echo "hello world"
$ git add .
$ git commit -m "First version script"
$ git push
```

Add Content

You can add content locally and push it to the server. You can build you first version of the script.

Demo

Create SSH Git Repo:

- Add git user
- Create ssh-keys
- Create empty repo
- Clone repo and add content

```
$ cd ~/project1
$ git status
$ git tag v1.0
$ git push origin v1.0
$ git tag
$ git show v1.0
$ rm -rf ~/project1
$ git clone -b v1.0 git@localhost:~/git/project1
```

Versioning and Tags

As your script is at a working level, however basic, you can save this with a tag. A tag, becomes a frozen release that is the end of development for that release. The code continues but heading towards the next milestone

Demo

Version Release:

- Using tags we create version releases

```
$ cd ~/project1
$ git checkout -b dev
$ vim hello.sh
#!/bin/bash
echo "hello world"
$ git add .
$ git commit -m "Add shebang to script"
$ git push origin dev
```

Branches and Development

When you are developing code, you should create a new branch and development. This preserves the integrity of the master branch. When the code is ready the development branch can be merged into the master branch.

Three Code Versions

master

Production Version

dev

Development

v1.0

Frozen Release

```
$ git status
$ git checkout master
$ git merge dev
$ git push
$ git tag v1.1
$ git push origin v1.1
```

Merging Development Branches with Master

Once a development cycle is complete, typically it will be thoroughly tested and merged into the master branch. This now becomes the production version. Optionally, a frozen release version may be created before further development

Demo

Development of Code:

- Released versions, tags
- Branches, development code
- Merging branches

Summary

Reviewed the course contents:

- Git
- Systemd units
- OpenSSL

Working with git

- Cloning public repos
- Creating local repos
- Creating SSH repos
- Using tags
- Using branches

Up Next:

Creating and Managing Unit Files in Systemd